# Introduction to Coding Theory

Tomasz Brengos
Committers : Aliaksei Kudzelka, Mykhailo Moroz

## 1 Encoding and Decoding Functions

Let $p$ be a prime number. We work over the finite field $\mathbb{Z}_p$ (the integers mod $p$). For any positive integer $n$, the set $\mathbb{Z}_p^n$ denotes the set of all $n$-tuples of elements from $\mathbb{Z}_p$. Note that since $\mathbb{Z}_p$ is a field, the spaces $\mathbb{Z}_p^n$ and $\mathbb{Z}_p^k$ can be viewed as vector spaces (of dimension $n$ or $k$ respectively) over $\mathbb{Z}_p$.

**Encoding function:** A map $\xi : \mathbb{Z}_p^k \to \mathbb{Z}_p^n$ is called an *encoding function* (or *coding function*) if it is one-to-one (distinct messages have distinct codewords). In other words, $\xi$ takes each possible message (an element of $\mathbb{Z}_p^k$) and encodes it as a longer string (an element of $\mathbb{Z}_p^n$). The image of $\xi$, denoted

$$\mathcal{C} = \xi(\mathbb{Z}_p^k) \subseteq \mathbb{Z}_p^n,$$

is the set of all possible encoded messages and is called the *code*. The elements of $\mathcal{C}$ are called *codewords*. Working over the field $\mathbb{Z}_p$ is crucial here, because it ensures we can use the tools of linear algebra and arithmetic modulo $p$ when designing and analyzing codes.

**Decoding function:** Given an encoding $\xi$, a map $\eta : \mathbb{Z}_p^n \to \mathbb{Z}_p^k$ is called a *decoding function* for $\xi$ if $\eta(\xi(u)) = u$ for every $u \in \mathbb{Z}_p^k$. In other words, $\eta$ is a (left) inverse of $\xi$ on the set of codewords, so that decoding an encoded message returns the original message (assuming no errors in transmission).

Once a message $u \in \mathbb{Z}_p^k$ is encoded as a codeword $c = \xi(u) \in \mathbb{Z}_p^n$, it is sent through a channel (transmission). During transmission, some components of $c$ might be altered due to noise or other errors.

## 2 Error Vectors and Transmission Errors

When a codeword is transmitted (over a noisy channel, or stored and later retrieved), some of its components may change due to errors. We can model the effect of errors by an *error vector*. An *error vector* $\mathbf{e} \in \mathbb{Z}_p^n$ is a vector that, when added to the original codeword, yields the *received word*. If $\mathbf{c} \in \mathcal{C}$ is the sent codeword and $\mathbf{e}$ is the error vector, then the received word is

$$\mathbf{r} = \mathbf{c} + \mathbf{e},$$

where addition is component-wise in $\mathbb{Z}_p$. (In the case $p = 2$, this addition is XOR of bits.) The nonzero entries of $\mathbf{e}$ indicate positions where an error occurred (and the value indicates what was added at that position, e.g. a flip from 0 to 1 in binary is represented by adding 1 mod 2).

The number of errors that occur is the number of nonzero entries in $\mathbf{e}$. This is called the *weight* of $\mathbf{e}$, often denoted $w(\mathbf{e})$. If $w(\mathbf{e}) = t$, we say $t$ errors occurred. The coding and decoding process can be summarized as:

$$\mathbf{u} \xrightarrow{\xi} \mathbf{c} \xrightarrow{\text{error } \mathbf{e}} \mathbf{r} = \mathbf{c} + \mathbf{e} \xrightarrow{\eta} \hat{\mathbf{u}},$$

where $\hat{\mathbf{u}} = \eta(\mathbf{r})$ is the decoder's estimate of the original message. We desire $\hat{\mathbf{u}} = \mathbf{u}$ even if $\mathbf{e}$ is nonzero (up to a certain weight). The design of $\mathcal{C}$ and $\eta$ determines how many errors can be reliably detected or corrected.

# 3 Examples of Encoding Functions (Codes)

We now present a few example encoding functions and their corresponding codes:

## Example 1: Repetition Code

One simple encoding is the *repetition code*. Here the message space is $\mathbb{Z}_p$ (messages of length 1). The encoding function $\xi_1 : \mathbb{Z}_p \to \mathbb{Z}_p^n$ for some chosen length $n$ is defined by

$$\xi_1(a) = (\underbrace{a, a, \ldots, a}_{n \text{ times}}),$$

i.e. the single-symbol message $a$ is repeated $n$ times to form the codeword. The code $\mathcal{C}_1 = \{(a, a, \ldots, a) : a \in \mathbb{Z}_p\}$ consists of all $n$-tuples with identical entries. This encoding is one-to-one (different $a$ give different constant tuples).

For example, over $\mathbb{Z}_2$ (the binary case), if $a = 1$ and $n = 5$, the codeword would be $(1, 1, 1, 1, 1)$. If a transmission error flips one of these bits (say we receive $(1, 1, 1, 0, 1)$), the decoder can notice that this is not a valid codeword in $\mathcal{C}_1$ (since not all bits are the same), hence an error is detected. In fact, the repetition code is able to detect up to $n - 1$ errors (any change in at most $n - 1$ positions will yield a tuple that is not constant and thus not in the code). This code even has error-correcting capability: intuitively, the decoder could guess that the majority value is the intended bit (for example, from $(1, 1, 1, 0, 1)$ a reasonable decoded value for $a$ would be 1, since most bits are 1). However, our focus here is on error detection criteria, which we formalize later.

## Example 2: Parity Check Code

Another useful encoding adds a *parity check* to the message. For this example, let's work in $\mathbb{Z}_p$ (and especially consider $p = 2$ for binary parity). Let the message space be $\mathbb{Z}_p^{k-1}$ (messages of length $k - 1$). Define an encoding function $\xi_2 : \mathbb{Z}_p^{k-1} \to \mathbb{Z}_p^k$ by

$$\xi_2(a_1, a_2, \ldots, a_{k-1}) = \big(a_1, a_2, \ldots, a_{k-1},\ a_k\big),$$

where the last component $a_k$ is chosen such that the total sum of the $k$ components is 0 in $\mathbb{Z}_p$. In other words,

$$a_k \ = \ -\left(a_1 + a_2 + \cdots + a_{k-1}\right) \pmod{p},$$

which ensures $a_1 + a_2 + \cdots + a_{k-1} + a_k \equiv 0 \pmod{p}$. This extra symbol $a_k$ is a redundancy (the *parity symbol*) chosen to enforce a constraint on every codeword.

The resulting code is

$$\mathcal{C}_2 = \{(a_1, \ldots, a_{k-1}, a_k) \in \mathbb{Z}_p^k : a_1 + \cdots + a_{k-1} + a_k = 0\}.$$

In the special case $p = 2$ (binary), this encoding is the standard single-parity-bit code: the last bit $a_k$ is 0 or 1 chosen to make the total number of 1's in the codeword even. For example, if the message is $(1, 0, 1, 1)$ in $\mathbb{Z}_2^4$ (so $k-1 = 4$), then the parity bit $a_5$ is chosen so that $1+0+1+1+a_5 \equiv 0 \pmod 2$. Here $1+0+1+1 = 3 \equiv 1 \pmod 2$, so we must take $a_5 = 1$ to make the sum even. The encoded 5-bit codeword is $(1, 0, 1, 1, 1)$, which has an even number of 1's (four 1's in this case).

This parity check code allows detection of any single-error in transmission: if one bit of a codeword is flipped, the parity (sum mod $p$) will no longer be zero, so the received word will violate the code constraint and thus be recognized as invalid. For instance, if $(1, 0, 1, 1, 1)$ is sent and one bit flips, the sum of bits will be odd, alerting us to the error. In fact, more generally, if an odd number of bits are in error, the parity check will detect it (sum becomes nonzero mod 2). However, if an even number of bits are corrupted in the binary case, the parity check could fail to detect it (since an even number of flips can restore the sum to even).

### Example 3: PESEL Check Digit (revised)

The Polish national identification number (PESEL) is an 11-digit string

$$YY\,MM\,DD\,PPPP\,K,$$

where

- $YYMMDD$ is the date of birth.
- $PPPP$ is a serial number; its last digit is *even* for females and *odd* for males.
- $K$ is the check digit making

$$\sum_{i=1}^{10} w_i\, p_i + K \equiv 0 \pmod{10}, \quad (w_i)_{i=1}^{10} = 1,3,7,9,1,3,7,9,1,3.$$

**Example.** A woman born on 15 April 1997 with serial 1478 has the preliminary digits 97 04 15 1478. The weighted sum is $1 \cdot 9 + 3 \cdot 7 + \cdots + 3 \cdot 8 = 156 \equiv 6 \pmod{10}$; hence $K = (10 - 6) \bmod 10 = 4$. The complete PESEL is `97041514784`.

**Error detection.** This checksum catches *every* single-digit error and the overwhelming majority of adjacent transposition errors.

For the first ten digits $(p_1, \ldots, p_{10}) \in \mathbb{Z}_{10}^{10}$ define

$$\xi_3 : \mathbb{Z}_{10}^{10} \longrightarrow \mathbb{Z}_{10}^{11}, \qquad \xi_3(p_1, \ldots, p_{10}) \;=\; \big(p_1, \ldots, p_{10},\, p_{11}\big),$$

where

$$p_{11} \;=\; \big(10 - \sum_{i=1}^{10} w_i p_i\big) \bmod 10, \quad (w_i)_{i=1}^{10} = 1,3,7,9,1,3,7,9,1,3.$$

Because $\xi_3$ is injective, its image $\mathcal{C}_3 = \xi_3(\mathbb{Z}_{10}^{10}) \subset \mathbb{Z}_{10}^{11}$ is a legitimate code—the PESEL code—whose distance properties are governed by the checksum just illustrated.

# 4 Hamming Distance

We need a way to quantify how "different" two vectors (e.g. the transmitted and received words) are. The appropriate notion is the *Hamming distance*. For vectors $\mathbf{x}, \mathbf{y} \in \mathbb{Z}_p^n$, the **Hamming distance** $d_H(\mathbf{x}, \mathbf{y})$ is defined as the number of coordinates in which $\mathbf{x}$ and $\mathbf{y}$ differ. Equivalently,

$$d_H(\mathbf{x}, \mathbf{y}) = |\{\, i : 1 \le i \le n,\ x_i \ne y_i \,\}|.$$

For example, in binary, $d_H(01011, 01101) = 2$ (they differ in two bit positions).

It is easy to check that Hamming distance satisfies the properties of a distance metric on $\mathbb{Z}_p^n$. In particular, for any $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \mathbb{Z}_p^n$:

1. $d_H(\mathbf{x}, \mathbf{y}) \ge 0$, and $d_H(\mathbf{x}, \mathbf{y}) = 0$ if and only if $\mathbf{x} = \mathbf{y}$.
2. $d_H(\mathbf{x}, \mathbf{y}) = d_H(\mathbf{y}, \mathbf{x})$ (symmetry).
3. $d_H(\mathbf{x}, \mathbf{z}) \le d_H(\mathbf{x}, \mathbf{y}) + d_H(\mathbf{y}, \mathbf{z})$ (triangle inequality).

Property (3) means that for any two vectors $\mathbf{x}$ and $\mathbf{z}$, any differences between $\mathbf{x}$ and $\mathbf{z}$ must show up when comparing $\mathbf{x}$ to some intermediate $\mathbf{y}$ or $\mathbf{y}$ to $\mathbf{z}$. In other words, any coordinate where $\mathbf{x}$ and $\mathbf{z}$ differ is either a coordinate where $\mathbf{x}$ differs from $\mathbf{y}$ or $\mathbf{y}$ differs from $\mathbf{z}$ (or both). This ensures no "shortcut" can make $d_H$ violate the triangle inequality. Thus, $d_H$ is a valid distance function.

The Hamming distance gives us a natural way to describe the effect of errors: if a codeword $\mathbf{c}$ is transmitted and a word $\mathbf{r}$ is received, then $d_H(\mathbf{c}, \mathbf{r}) = w(\mathbf{e})$, the weight of the error vector (the number of errors that occurred). For a given code $C \subseteq \mathbb{Z}_p^n$, we often speak of the distance *between codewords* measured by $d_H$. The **minimum distance** of code $C$, denoted $d(C)$ or $d_C$, is the smallest Hamming distance between any two distinct codewords in $C$:

$$d_C := \min\{\, d_H(\mathbf{c}_1, \mathbf{c}_2) : \mathbf{c}_1, \mathbf{c}_2 \in C,\ \mathbf{c}_1 \neq \mathbf{c}_2 \,\}.$$

# 5 Metric Spaces and Other Distance Metrics

The pair $(\mathbb{Z}_p^n, d_H)$ is an example of a *metric space*. In general, a **metric space** is a tuple $(X, d)$, where $X$ is a set and

$$d : X \times X \longrightarrow \mathbb{R}$$

is a function (called a *metric*) satisfying the three properties listed above (non-negativity and identity of indiscernibles, symmetry, and the triangle inequality). The value $d(x, y)$ represents the "distance" between points $x$ and $y$ in the space.

There are many examples of metrics aside from the Hamming distance. A few common metrics are:

- **Euclidean distance.** On $\mathbb{R}^n$,

$$d_E(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2},$$

  the usual "straight–line" distance from geometry derived from the Pythagorean theorem.

- **Maximum (Chebyshev) distance.** Also on $\mathbb{R}^n$ or $\mathbb{Z}^n$,

$$d_\infty(\mathbf{x}, \mathbf{y}) = \max_{1 \leq i \leq n} |x_i - y_i|.$$

  Here distance is determined by the single largest coordinate-difference.Intuitively, the distance is measured in terms of the largest single-coordinate difference. For example, in the plane $\mathbb{R}^2$, the distance between $(x_1, y_1)$ and $(x_2, y_2)$ is $\max(|x_2 - x_1|,\ |y_2 - y_1|)$, which in effect produces a square-shaped notion of distance (all points with $d_\infty$ distance $\leq r$ from a center form a square of side length $2r$).

Many other metrics exist (Manhattan distance, discrete metric, etc.), but the Hamming distance is particularly useful for analyzing codes. We will now focus on metric properties in the context of coding theory.

# 6 Balls and Error Correction

Given a metric space $(X, d)$, a **ball** (or *distance ball*) of radius $r$ centered at a point $x \in X$ is the set

$$B_r(x) = \{\, y \in X : d(x, y) \leq r \,\}.$$

It contains all points that are at distance at most $r$ from $x$. In the Hamming metric space $(\mathbb{Z}_p^n, d_H)$, the ball of radius $r$ around a vector $\mathbf{x}$ consists of all vectors that differ from $\mathbf{x}$ in at most $r$ coordinate positions. For example, in $(\mathbb{Z}_2)^3$ (3-bit binary space),

$$B_1(000) = \{000,\ 001,\ 010,\ 100\},$$

since these are exactly the 3-bit vectors with at most 1 bit different from 000. The size of $B_r(x)$ in a Hamming space depends on $r$ and $n$; for instance, in $(\mathbb{Z}_2)^3$, a radius-1 ball has $1 + \binom{3}{1} = 4$ elements (including the center).

In coding theory, metric balls are a useful way to visualize and implement error correction. Suppose we have a code $C \subseteq \mathbb{Z}_p^n$. To *correct up to $t$ errors*, the decoding process can be viewed as follows: given a received word $\mathbf{r}$, find the codeword $\mathbf{c} \in C$ that is closest to $\mathbf{r}$ in Hamming distance (i.e. with minimal $d_H(\mathbf{c}, \mathbf{r})$). If the number of errors is at most $t$, and if certain distance conditions hold, this closest codeword will be the one that was originally sent. In essence, we imagine around each codeword $\mathbf{c}$ a *ball of radius $t$*. If no two codewords' radius-$t$ balls overlap, then any received word that lies within distance $t$ of a codeword $\mathbf{c}$ could only have come from $\mathbf{c}$ (because if it were also within $t$ of a different codeword $\mathbf{c}'$, then the balls would intersect at $\mathbf{r}$). In that case, we can unambiguously decode $\mathbf{r}$ to $\mathbf{c}$. On the other hand, if the balls of radius $t$ around codewords overlap or touch, a received word could be close to two different codewords, causing ambiguity in decoding.

Metric balls also give a criterion for *detecting* errors (even if we cannot correct them). A code can *detect* up to $t$ errors if whenever up to $t$ errors occur, the result is *not* a valid codeword (unless no error occurred). In terms of distance, this means that if you take any codeword $\mathbf{c}$ and look at all vectors within distance $t$ of $\mathbf{c}$ (excluding $\mathbf{c}$ itself), none of those vectors are codewords in $C$. Equivalently, the balls of radius $t$ around each codeword $\mathbf{c} \in C$, *not counting the center*, do not contain any other codeword. (They may overlap with balls around other codewords, but only at non-codeword points, which corresponds to errors causing an invalid sequence that the receiver can recognize as erroneous.)

# 7 A Geometric Example: $(\mathbb{Z}_2)^3$ as a Hamming Space

To visualize these concepts, consider the 3-dimensional binary vector space $(\mathbb{Z}_2)^3$ with the Hamming metric. This space consists of 8 vectors (from 000 to 111). We can geometrically represent $(\mathbb{Z}_2)^3$ as the vertices of a cube, where edges connect vectors that differ in exactly one coordinate (Hamming distance 1 apart). In the figure below, each vertex is labeled by the corresponding 3-bit vector, and each edge represents a single-bit flip. This graph is often called the 3-dimensional *Hamming cube*.

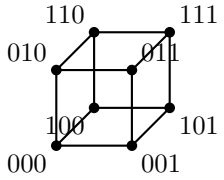



Figure 1: Each vertex corresponds to a 3-bit binary vector. Edges connect vertices that differ in one coordinate (Hamming distance 1). For example, the neighbors of 000 (connected directly by an edge) are 001, 010, and 100, which illustrates all vectors at Hamming distance 1 from 000.

In this cube, one can see how clusters of vectors can be grouped around a given vertex. For instance, as noted, the set $\{000, 001, 010, 100\}$ are all within distance 1 of 000 (a Hamming ball of radius 1 around 000). If we take a simple code such as $C = \{000, 111\}$ (which is the $((3, 1))$ repetition code in binary with $n = 3$, $k = 1$), those codewords correspond to the vertices 000 and 111, which are opposite corners of the cube. The minimum distance of this code is $d_C = d_H(000, 111) = 3$. The balls of radius 1 around 000 and around 111 (the sets of neighbors each has) do not overlap; in fact, they are separated by at least one bit flip difference. Thus, if at most 1 error occurs during transmission, the received word will remain in the unique "sphere" of radius 1 around whichever codeword was sent, and decoding can correctly deduce the original codeword. If 2 errors occur, the received word will be distance 2 from the original codeword — in this case (for $C = \{000, 111\}$) a double error could yield a vector like 110, which is not a codeword but is now only distance 1 away from the *other* codeword 111. The decoder might misidentify it if it only considers radius 1

spheres, but it will at least notice an error occurred since 110 is not itself a codeword.

# 8 Error Detection and Correction Bounds

Before looking at concrete codes, it is helpful to understand in general how the minimum distance of a code determines its ability to detect and correct errors.

**1. Minimum distance.** Let $C \subseteq \mathbb{Z}_p^n$ be a code with minimum Hamming distance

$$d_{\min} = \min_{\substack{\mathbf{c}_1, \mathbf{c}_2 \in C \\ \mathbf{c}_1 \neq \mathbf{c}_2}} d_H(\mathbf{c}_1, \mathbf{c}_2).$$

By definition, any two distinct codewords differ in at least $d_{\min}$ positions.

**2. Error detection.** If up to $t$ errors occur, then the received word $\mathbf{r} = \mathbf{c} + \mathbf{e}$ satisfies

$$d_H(\mathbf{c}, \mathbf{r}) = w(\mathbf{e}) \leq t.$$

To *detect* every such error, $\mathbf{r}$ must never coincide with a different codeword. In other words, no two codewords may be within distance $t$ of one another. Since the closest pair of distinct codewords are $d_{\min}$ apart, we need

$$t < d_{\min}.$$

Hence a code with minimum distance $d_{\min}$ can detect all error patterns of weight up to

$$t_{\text{det}} = d_{\min} - 1.$$

**3. Error correction.** To *correct* up to $t$ errors, a decoder typically chooses the codeword nearest to the received word $\mathbf{r}$. Geometrically, this means the Hamming balls of radius $t$ around each codeword must be disjoint: if two balls overlap, a received word in the overlap could have come from either center, making correct decoding impossible.

Two balls of radius $t$ around codewords $\mathbf{c}_1$ and $\mathbf{c}_2$ remain disjoint exactly when

$$2t < d_H(\mathbf{c}_1, \mathbf{c}_2) \quad \text{for all } \mathbf{c}_1 \neq \mathbf{c}_2.$$

Since the minimum distance between any two distinct codewords is $d_{\min}$, the strongest requirement is

$$2t < d_{\min}.$$

Therefore, the maximum number of errors that can be *corrected* is

$$t_{\text{corr}} = \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor.$$

**Summary.** If a code $C$ has minimum Hamming distance $d_{\min}$, then

$$t_{\text{det}} = d_{\min} - 1, \quad t_{\text{corr}} = \left\lfloor \tfrac{d_{\min}-1}{2} \right\rfloor.$$

These simple formulas allow us, once we know $d_{\min}$, to immediately read off how many errors the code can detect and how many it can correct.

### Example 1

Consider the 5-bit binary code

$$C = \{\texttt{00000},\ \texttt{01011},\ \texttt{10101},\ \texttt{11110}\} \subset (\mathbb{Z}_2)^5.$$

Checking pairwise Hamming distances gives

$$d_H(00000, 01011) = 3, \quad d_H(00000, 10101) = 3, \quad d_H(01011, 10101) = 4, \ \ldots$$

so the minimum distance is

$$d_{\min} = 3.$$

Hence

$$t_{\text{det}} = 3 - 1 = 2, \quad t_{\text{corr}} = \lfloor (3-1)/2 \rfloor = 1.$$

*Conclusion:* this code can detect up to 2 errors and correct up to 1 error.

### Example 2

Now take the 6-bit code

$$C = \{\texttt{000000},\ \texttt{010101},\ \texttt{101010},\ \texttt{111111}\} \subset (\mathbb{Z}_2)^6.$$

One finds quickly that every pair of distinct codewords differs in exactly 3 or 6 positions, so

$$d_{\min} = 3.$$

Thus again

$$t_{\text{det}} = 2, \quad t_{\text{corr}} = 1.$$

### Example 3: Repetition Code of Length 7

Finally, the binary repetition code of length 7 is

$$C = \{\underbrace{0000000}_{7 \text{ times}}, \underbrace{1111111}_{7 \text{ times}}\}.$$

Clearly any two distinct codewords differ in all 7 positions, so

$$d_{\min} = 7.$$

Therefore

$$t_{\text{det}} = 7 - 1 = 6, \quad t_{\text{corr}} = \lfloor (7-1)/2 \rfloor = 3.$$

*Conclusion:* the 7-fold repetition code can detect up to 6 errors and correct up to 3 errors.

## 9 Detection, Correction, and Perfect Codes

### Theorem

*A code $C$ is able to detect $t$ errors or fewer* iff *the minimum distance satisfies $d_C \geq t + 1$,*

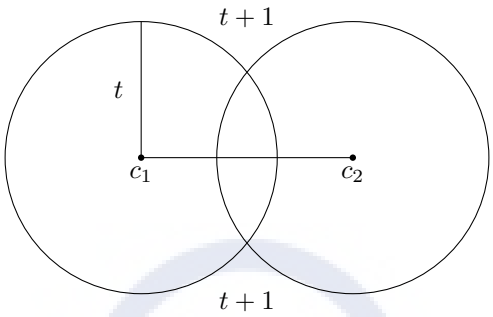
$$d_C = \min\{\, d_H(c_1, c_2) : c_1, c_2 \in C,\ c_1 \neq c_2 \,\}.$$

## Proof

**(⇒)** Let $c_1 \in C$ be the original codeword sent, and let $f$ be the received word. Assume $d_C \geq t+1$. If at most $t$ errors occurred, then $d_H(c_1, f) \leq t$. For any *other* codeword $c_2 \in C$ with $c_2 \neq c_1$ we have

$$d_H(c_2, f) \;\geq\; d_H(c_1, c_2) - d_H(c_1, f) \;\geq\; (t+1) - t \;=\; 1.$$

Hence $f \notin C$, so the decoder recognises that an error happened; the error is *detectable*.




**(⇐)** Suppose the code detects every pattern of at most $t$ errors. For any $c \in C$ and word $f$ with $d_H(c, f) \leq t$, we must have $f \notin C$ unless $f = c$. Thus if $c, c' \in C$ are distinct, $c'$ cannot lie within distance $t$ of $c$; otherwise $c'$ would be a second codeword in the ball of radius $t$ around $c$. Therefore $d_H(c, c') > t$ for all $c \neq c'$, hence (integer distance) $d_H(c, c') \geq t+1$. Taking the minimum over all pairs gives $d_C \geq t+1$. □

## Definition

A code $C$ *corrects $t$ errors or fewer* if, whenever $d_H(c, f) \leq t$ for some $c \in C$, that $c$ is the *unique* codeword within distance $\leq t$ of $f$.

## Theorem

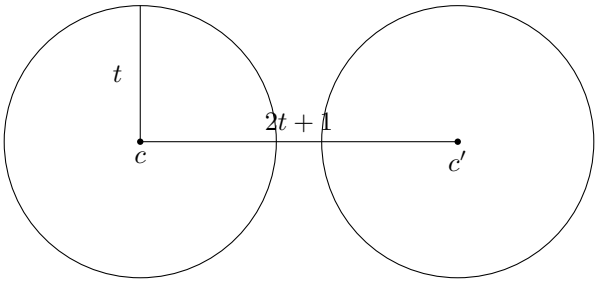
A code $C$ corrects $t$ or fewer errors *iff*

$$d_C \;\geq\; 2t+1.$$

## Corollary

$C$ corrects $t$ errors or fewer *iff*

$$B(c, t) \cap B(c', t) = \varnothing \quad \text{for all } c \neq c' \in C.$$



## Example

Let

$$\varphi :\ \mathbb{Z}_p^{\,n-1} \longrightarrow \mathbb{Z}_p^{\,n}, \qquad \varphi(a_1, \ldots, a_{n-1}) \ = \ (a_1, \ldots, a_{n-1}, -\sum_{i=1}^{n-1} a_i).$$

This is the length-$n$ *single-parity-check code*. Its minimum distance is $d_C = 2$.

- **Detection:** $d \geq t + 1$ $(2 \geq t + 1 \Rightarrow t \leq 1)$. So the code *detects exactly one error*.
- **Correction:** $d \geq 2t + 1$ $(2 \geq 2t + 1$ impossible for $t \geq 1)$. Hence it *cannot correct a single error* (only $t = 0$).

## Example (repetition code)

Define

$$\xi : \mathbb{Z}_p \longrightarrow \mathbb{Z}_p^{\,n}, \qquad \xi(a) = (\underbrace{a, \ldots, a}_{n \text{ times}}).$$

This $n$-fold repetition code has

$$d_C = n, \qquad \text{detection capacity } n - 1, \qquad \text{correction capacity } \left\lfloor \frac{n-1}{2} \right\rfloor.$$

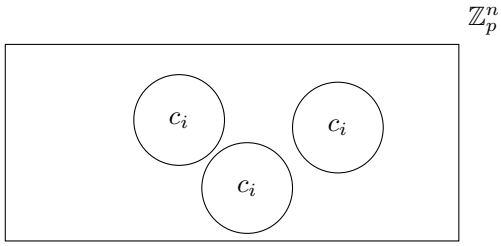
## Theorem (Hamming bound)

If a code $C \subseteq \mathbb{Z}_p^{\,n}$ with $M$ elements corrects $t$ errors, then

$$M\big(1 + (p-1)\tbinom{n}{1} + (p-1)\tbinom{n}{2} + \cdots + (p-1)\tbinom{n}{t}\big) \ \leq \ p^{\,n}.$$

### Proof (sketch)

Draw a radius-$t$ Hamming ball around each codeword. Since the balls are disjoint, their union fits inside $\mathbb{Z}_p^{\,n}$, giving the inequality.




## Definition (perfect code)

A code correcting $t$ errors is *perfect* when equality holds, i.e.

$$M\big(1 + (p-1)\tbinom{n}{1} + \cdots + (p-1)\tbinom{n}{t}\big) = p^{\,n}.$$

# 10 Linear Codes

## Definition (Linear codes)

Let $\zeta : \mathbb{Z}_p^k \to \mathbb{Z}_p^n$ be a coding function that maps a message vector of length $k$ to a codeword of length $n$ (where $k < n$). The function $\zeta$ is called a linear encoder, if it is a linear mapping between the vector spaces $\mathbb{Z}_p^k$ and $\mathbb{Z}_p^n$. That is, for all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_p^k$ and all scalars $\alpha, \beta \in \mathbb{Z}_p$, the function satisfies:

$$\zeta(\alpha\mathbf{u} + \beta\mathbf{v}) = \alpha\zeta(\mathbf{u}) + \beta\zeta(\mathbf{v})$$

The image of $\mathbb{Z}_p^k$ under $\zeta$, denoted as $C = \mathrm{Im}(\zeta)$, is a subspace of $\mathbb{Z}_p^n$.

Hence: A non-empty subset $C \subseteq \mathbb{Z}_p^n$ is a linear code if and only if it is closed under linear combinations. That is, for all codewords $\mathbf{c}_1, \mathbf{c}_2 \in C$ and all scalars $\alpha, \beta \in \mathbb{Z}_p$:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 \in C$$

## Example 1

Consider the repetition code

$$C = \left\{ \underbrace{(a, a, \ldots, a)}_{n \text{ times}} \;\middle|\; a \in \mathbb{Z}_p \right\} \subseteq \mathbb{Z}_p^n$$

It is easy to show that $C$ is a linear code.

Proof:
To prove that $C$ is a linear code, we must demonstrate that it is a subspace of $\mathbb{Z}_p^n$. We will use the combined subspace criterion. Let $\mathbf{c}_1, \mathbf{c}_2$ be any two codewords from $C$, and let $\alpha, \beta$ be any two scalars from the finite field $\mathbb{Z}_p$. By definition of the set $C$, since $\mathbf{c}_1 \in C$ and $\mathbf{c}_2 \in C$, there must exist two scalars $a_1, a_2 \in \mathbb{Z}_p$ such that:

$$\mathbf{c}_1 = (a_1, a_1, \ldots, a_1)$$
$$\mathbf{c}_2 = (a_2, a_2, \ldots, a_2)$$

Now, let us construct the linear combination $\alpha\mathbf{c}_1 + \beta\mathbf{c}_2$:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 = \alpha(a_1, a_1, \ldots, a_1) + \beta(a_2, a_2, \ldots, a_2)$$

Using component-wise vector addition and scalar multiplication in $\mathbb{Z}_p^n$, we can combine these terms:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 = (\alpha a_1, \alpha a_1, \ldots, \alpha a_1) + (\beta a_2, \beta a_2, \ldots, \beta a_2)$$

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 = (\alpha a_1 + \beta a_2,\ \alpha a_1 + \beta a_2,\ \ldots,\ \alpha a_1 + \beta a_2)$$

Let us define a new scalar $a_3 = \alpha a_1 + \beta a_2$. Since $\mathbb{Z}_p$ is a field, it is closed under addition and multiplication, meaning that $a_3 \in \mathbb{Z}_p$. Substituting $a_3$ back into our vector, we get:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 = \underbrace{(a_3, a_3, \ldots, a_3)}_{n \text{ times}}$$

Since this resulting vector consists of the same scalar $a_3$ repeated $n$ times, where $a_3 \in \mathbb{Z}_p$, it perfectly matches the definition of the set $C$. Therefore:

$$\alpha\mathbf{c}_1 + \beta\mathbf{c}_2 \in C$$

Because the linear combination of any two codewords remains within $C$, the set $C$ is a valid subspace of $\mathbb{Z}_p^n$. Hence, $C$ is a linear code. □

## Example 2

Formally, the general parity-check code $C$ is defined as the following set:

$$C = \left\{ \left( a_1, a_2, \ldots, a_{n-1}, -\sum_{i=1}^{n-1} a_i \right) \;\middle|\; a_i \in \mathbb{Z}_p \right\} \subseteq \mathbb{Z}_p^n$$

Proof:
Let $\mathbf{c} = (c_1, c_2, \ldots, c_n)$ be an arbitrary vector in $\mathbb{Z}_p^n$. By definition, $\mathbf{c}$ belongs to $C$ if and only if its components satisfy the relation:

$$c_n = -\sum_{i=1}^{n-1} c_i$$

By moving the summation to the left-hand side of the equation, we can rewrite this structural constraint as a single homogeneous linear equation:

$$c_1 + c_2 + \cdots + c_{n-1} + c_n = 0$$

Thus, the code $C$ can be explicitly defined as the set of all vectors that satisfy this homogeneous equation:

$$C = \left\{ \mathbf{c} \in \mathbb{Z}_p^n \;\middle|\; \sum_{i=1}^{n} c_i = 0 \right\}$$

From fundamental linear algebra, the set of all solutions to any system of homogeneous linear equations over a field always forms a vector subspace of the surrounding vector space. □

## Remark

Let
$\zeta : \mathbb{Z}_p^k \to \mathbb{Z}_p^n$ be a linear encoding function defining a linear code $C$. Then, there exists a $k \times n$ matrix $G$ with entries in $\mathbb{Z}_p$ such that for every message $\mathbf{u} \in \mathbb{Z}_p^k$, the encoding function can be presented as:

$$\zeta(\mathbf{u}) = \mathbf{u}G$$

The matrix $G$ is called the **generator matrix** of the linear code $C$.

## Remark

Let $C$ be a linear code and let $G$ be its generating matrix. Then, every codeword $\mathbf{c} \in C$ can be expressed as a linear combination of the rows of $G$.

$$\mathbf{c} \in C \iff \mathbf{c} = \mathbf{u}G \quad \text{for } \mathbf{u} \in \mathbb{Z}_p^k$$

Rephrased:

$$C = \left\{ \mathbf{u}G \;\middle|\; \mathbf{u} \in \mathbb{Z}_p^k \right\}$$

## Example 3

$$C = \{(0,0,0,0), (0,1,0,1), (1,0,1,1), (1,1,1,0)\} \subseteq \mathbb{Z}_2^4$$

C is a linear code, since it is closed under linear combinations. Encoding function:

$$\zeta : \mathbb{Z}_2^2 \to \mathbb{Z}_2^4$$

Generating matrix:

$$G = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

$$(0,0)G = (0,0,0,0) \in C$$

$$(0,1)G = (1,0,1,1) \in C$$

$$(1,0)G = (0,1,0,1) \in C$$

$$(1,1)G = (1,1,1,0) \in C$$

Notice that G is not unique. For instance, we could also use the following generator matrix:

$$G' = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

or

$$G'' = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

The rows of the generator matrix $G$ must be linearly independent.

## Definition (standard form)

We say that a generating matrix $G$ is in **standard form** if is of the form:

$$G = [I_k|P]$$

where $I_k$ is the $k \times k$ identity matrix and $P$ is a $k \times (n-k)$ matrix.

## Remark

If G is in standard form, then:

$$\underbrace{(u1, u2, ..., uk)}_{\text{u}} G = (u1, u2, ..., uk, ...)$$

## Example

$$C = \left\{ \left( a_1, a_2, \ldots, a_{n-1}, -\sum_{i=1}^{n-1} a_i \right) \;\middle|\; a_i \in \mathbb{Z}_p \right\}$$

A generator matrix for this code in standard form is:

$$G = \left[\begin{array}{cccccc} 1 & 0 & 0 & \ldots & 0 & -1 \\ 0 & 1 & 0 & \ldots & 0 & -1 \\ 0 & 0 & 1 & \ldots & 0 & -1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \underbrace{0 \quad 0 \quad 0 \quad \ldots \quad 1}_{I_{n-1}} & & & & & -1 \end{array}\right]$$

$$(a_1, a_2, \ldots, a_{n-1})G = (a_1, a_2, \ldots, a_{n-1}, -\sum_{i=1}^{n-1} a_i)$$

Notice that if we define H = (1, 1, ..., 1), then:

$$\mathbf{c}H^T = 0 \iff \mathbf{c} \in C$$

## Definition (parity-check matrix)

Let $C$ be a linear code in $\mathbb{Z}_p^n$. with generating matrix $G$ of size $k \times n$. A matrix $H$ of size $(n-k) \times n$ over $\mathbb{Z}_p$ is called a **parity-check matrix** for $C$, whenever:

$$\mathbf{c} \in C \iff \mathbf{c}H^T = 0$$

## How to obtain H from G?

If $G$ is in standard form, i.e. $G = [I_k|P]$, then a parity-check matrix for $C$ can be constructed as:

$$H = [-P^T|I_{n-k}]$$

H is parity check matrix for $C$ because:

$$\mathbf{c} \in C \iff \mathbf{c} = \mathbf{u}G \text{ for some } \mathbf{u} \in \mathbb{Z}_p^k$$

Now,

$$\mathbf{c}H^T = \mathbf{u}GH^T = \mathbf{u}[I_k|P][-P^T|I_{n-k}]^T = \mathbf{u}[I_k|P]\begin{bmatrix} -P \\ I_{n-k} \end{bmatrix} = \mathbf{u}(-P+P) = \mathbf{0}$$

If the generating matrix $G$ is not in standard form, the parity-check matrix $H$ can be constructed by solving the homogeneous system of linear equations:

$$G\mathbf{x} = \mathbf{0}$$

Since the solution set $V = \{\mathbf{x} \in \mathbb{Z}_p^n : G\mathbf{x}^T = \mathbf{0}^T\}$ forms a subspace of $\mathbb{Z}_p^n$ of dimension $n-k$, it possesses a basis. Let $\{\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_{n-k}\}$ be a basis of this solution space. The parity-check matrix $H$ is then formed by taking these basis vectors as its rows:

$$H = \begin{bmatrix} \mathbf{h}_1 \\ \mathbf{h}_2 \\ \vdots \\ \mathbf{h}_{n-k} \end{bmatrix}$$

By construction, $GH^T = \mathbf{0}$, which ensures that $H$ is a valid parity-check matrix for the code $C$.

## Decoding linear codes

Assume C is a linear code with parity-check matrix H. Moreover assume $\mathbf{c}$ is the codeword sent, but $\mathbf{r} = \mathbf{c} + \mathbf{e}$ was the received word, where $\mathbf{e}$ is the error vector. Then:

$$\mathbf{r}H^T = (\mathbf{c}+\mathbf{e})H^T = \mathbf{c}H^T + \mathbf{e}H^T = \mathbf{0} + \mathbf{e}H^T = \mathbf{e}H^T$$

The product $\mathbf{s} = \mathbf{r}H^T$ is called the syndrome of the received vector $\mathbf{r}$.

## Theorem

If C corrects $d$ errors or less and $\mathbf{r} = \mathbf{c} + \mathbf{e}$, where $\mathbf{c} \in C$ is the codeword sent and $\mathbf{e}$ is the error vector, whose weight is at most $d$, then $\mathbf{e}H^T$ is unique and can be determined by finding all solutions to:

$$\mathbf{r}H^T = \mathbf{x}H^T$$

and choosing the solution $\mathbf{x}$ with the smallest weight.

### Example

Let C = $\{(0,0,0,0),(0,1,0,1),(1,0,1,1),(1,1,1,0)\}$ be a linear code with generating matrix:

$$G = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

and parity-check matrix:

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

Suppose the received word is $\mathbf{r} = (1,1,1,1)$.

$$rH^T = (1,1,1,1)\begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} = [0,1] \text{ in } \mathbb{Z}_2^2$$

The solutions to $\mathbf{r}H^T = \mathbf{x}H^T$ are:

$$\mathbf{x} = (x_3, 1 + x_3 + x_4, x_3, x_4) \quad \text{for } x_3, x_4 \in \mathbb{Z}_2$$

Vectors from solution set with the smallest weight are:

$$\mathbf{e}_1 = (0,1,0,0)$$

$$\mathbf{e}_2 = (0,0,0,1)$$

Hence, the decoder can choose either $\mathbf{e}_1$ or $\mathbf{e}_2$ as the error vector, and decode $\mathbf{r}$ to $\mathbf{c} = \mathbf{r} - \mathbf{e}_1 = (1,0,1,1)$ or $\mathbf{c} = \mathbf{r} - \mathbf{e}_2 = (1,1,1,0)$, both of which are valid codewords in $C$.